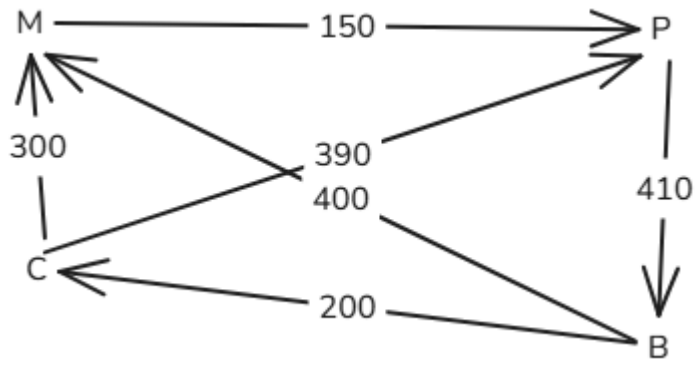
M = Montevideo, C = Colonia, B = Buenos Aires, P = Punta del Este



Floyd tiene orden $O(n^3)$, porque toma cada uno de los n vértices como intermedio y, para cada uno, revisa todos los pares origen–destino. Además necesita $O(n^2)$ de memoria para las matrices de costos y de predecesores.

¿Se puede reducir? Si el grafo es denso, no hay una mejora real. Si el grafo es disperso, se puede ejecutar Dijkstra desde cada vértice, usando una cola de prioridad, con un costo aproximado de $O(n \cdot e \log n)$.

Impacto en este problema: como hay solamente cuatro ciudades, Floyd realiza $4^3 = 64$ comparaciones en su ciclo más interno. Por eso, el costo es muy bajo. Además, las distancias se calculan una sola vez y luego cada consulta se responde en O(1).

A_0

	M	**C**	**B**	**P**
M	0	∞	∞	150
C	300	0	∞	390
B	400	200	0	∞
P	∞	∞	410	0

k = M: B→P = 400 + 150 = 550.
k = C: sin cambios.
k = B: P→M = 410 + 400 = 810; P→C = 410 + 200 = 610.

k = P: M→C = 150 + 610 = 760; M→B = 150 + 410 = 560; C→B = 390 + 410 = 800.

A final

	M	**C**	**B**	**P**
M	0	760	560	150
C	300	0	800	390
B	400	200	0	550
P	810	610	410	0

En Punta del Este, porque es el centro del grafo: tiene la menor excentricidad (el máximo de su columna en A final). Desde cualquier ciudad se llega a Punta del Este con un costo de a lo sumo 550.

	M	**C**	**B**	**P**
Excentricidad	810	760	800	**550**

```
imprimirCamino(i, j)
  imprimir i
  imprimirIntermedios(i, j)
  imprimir j

imprimirIntermedios(i, j)
  k ← P[i][j]
  si k es nulo entonces retornar
  imprimirIntermedios(i, k)
  imprimir k
  imprimirIntermedios(k, j)
```